

Prof. Dr. Stefan Leutenegger

Teaching Assistants: Tianyi Zhang, Cédric Le Gentil, and Simon Boche

Exercise 5: Camera

Session date: 31/03/2026

Task 1: Camera Projection and Back-Projection (Python)

You are supposed to implement the camera projection and back-projection functions of a pinhole camera model with radial-tangential distortion into the Python script `e05_pinhole_camera.py`.

For installation of required packages for this and the next task, we provide you with a `requirements.txt`. You can install them using e.g. `pip install -r requirements.txt`

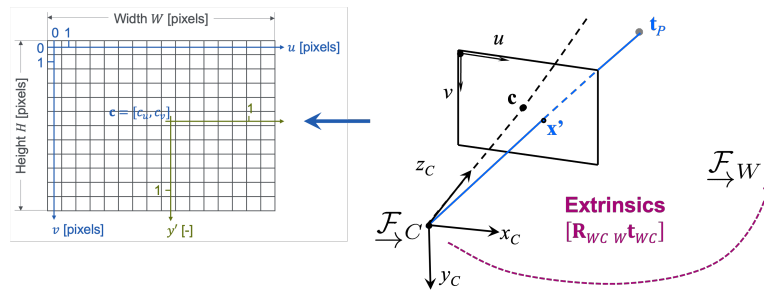


Figure 1: Coordinate System Convention.

- Before the Python implementation, let's first review the concepts with a simple camera projection and back-projection example. Consider a camera with intrinsic parameters $f_u = f_v = 100$, $c_u = 320$, $c_v = 240$, radial distortion parameters $k_1 = 0.2$, $k_2 = k_3 = k_4 = k_5 = k_6 = 0$, and tangential distortion parameters $p_1 = 0.01$ and $p_2 = 0.01$. ${}^C\mathbf{t}_P = [1, 1, 2]^T$ is a 3D point in camera coordinates as seen in Figure 1. Compute the 2D projection $\mathbf{u}$ of point ${}^C\mathbf{t}_P$ in pixel coordinates.
- Study the code: note how the necessary parameters are part of the class `pinholeCamera` and how you are meant to split the function `project(.)` into functions `p(.)`, `d(.)`, and `k(.)` as introduced in the lecture. Also note the use of the `RadialTangentialDistortion` helper class, which could now be easily swapped for a different distortion model.
- Implement the `project(.)` function. Make sure to return an empty array, if the projection is invalid. Further implement the generation of the 3D cube and qualitatively check if the projected image matches your expectation of the distortion model.
- Implement the `backProject(.)` function. Note the readily available `d_inverse(.)` function (since it requires a numeric solver, we implemented this for you, but be aware that it will use your implementation of `distort(.)` in the `RadialTangentialDistortion` class to do so).
- To really make sure you did everything correctly, implement a unit test that generates a random visible 3D point from a random image coordinate using the `backProject(.)` function (but also at a random depth), and then projects it into the image using `project(.)` – where the projection should then match the originally created image point.

- vi) Visualize and compare using real data. A ground-truth depth map and a depth map estimated by a state-of-the-art monocular network Metric3Dv2¹ is provided. Qualitatively compare them by using the above `PinholeCamera` class to back-project the images into 3D with the given depth maps and visualize the 3D point clouds. Then quantitatively compare them by computing the average absolute relative depth error

$$\delta_{rel} = |d_i^{est} - d_i^{gt}| / d_i^{gt} \quad (1)$$

In particular, you can find the image, camera intrinsics, and depth maps in the `./data/pinhole/` folder. The folder includes the following files:

- 2D image: `rgb.jpg`.
- Camera intrinsic: `intrinsic.txt`. We assume a pinhole camera model without distortion here.
- Depth maps: `depth_gt.npy` stores the ground-truth depth and `depth_est.npy` contains the estimated depth.

Task 2: Stereo Vision (Python)

This exercise considers the stereo triangulation problem. You will see how a stereo camera setup can be used to recover depth information from images in the Python script `e05_stereo_vision.py`. In this section, we assume rectified stereo camera setup without distortion.

- Before the Python implementation of the following tasks, let's first work through a written example. Consider rectified stereo cameras with focal length $f = 800$, principal point $(c_u, c_v) = (320, 240)$, and baseline $b = 0.2$ m.
 - A 3D point is observed at pixel $u_l = 450$ in the left image and $u_r = 370$ in the right image. The vertical image coordinate of the point is $v = 300$. Compute the disparity d , depth z , and determine the 3D coordinates ${}_C \mathbf{t}_P$ in the left camera coordinate.
 - Consider that the disparity measurement is affected by noise of $\Delta d = 1$ pixel. Derive the depth uncertainty Δz as a function of depth. Then use this expression to compute the depth uncertainty for the above setting. Next, change the baseline b to 0.1 m and 0.4 m and recompute the depth uncertainty. Briefly explain how the accuracy of stereo depth estimation can be improved.
- Stereo projection. We provide two cameras in a given stereo setup together with the Utah teapot point cloud. Your first task is to project the 3D points into the two image planes, create the corresponding projection images, and visualize them. For simplicity, you may import the `PinholeCamera` class from Task 1. Note that you need to transform the points into the corresponding camera frame before projection.
- Compute the disparity for all valid projected points. Visualize the disparity values as a color map using the readily available function `plotDisparity()`.
- Reconstruct the 3D structure from the disparity map. Complete the function `stereoTriangulation()` and visualize the recovered 3D structure.
- Add white noise to the disparity map using the helper function `addNoise()`, visualize the triangulated 3D result, and study how the stereo baseline influences the reconstruction quality.
- Triangulate the real-world data using the triangulation function above. We provide both the ground-truth disparity map and the disparity map estimated by FoundationStereo². Similar to Task 1, compute the average absolute relative depth error, visualize the point clouds, and compare them both qualitatively and quantitatively. You can find the data in the `./data/stereo/` folder

¹<https://github.com/YvanYin/Metric3D>

²<https://github.com/NVlabs/FoundationStereo>

- Disparity maps: `disp_gt.npy` is the ground-truth disparity map and `disp_est.npy` is the predicted disparity map.
- Camera setup: you can find the camera setup in `calib.txt`.